Text-to-Image Generation with Stable Diffusion and CLIP

Team: Gokul Chandar Kumaran, Loalith Madala, Yagna Praneeth Reddy Panguru · November 2026 **Repo:** github.com/GokulChandarKumaran/text2image-diffusion

The idea

Image generators are great at making pretty pictures, but they don't always draw what you asked for. When we tested Stable Diffusion with "two cats sleeping on a blue couch next to a remote control," we got the cats and the couch, but no remote. We asked for a double-decker bus and got a regular one. Small misses like these are what we want to work on. Our plan is to measure how well a pretrained model sticks to its prompt, then fine-tune it on a cleaned-up set of real captioned photos and see whether it gets better. If the fine-tuned model scores higher than the original, and people actually prefer its pictures when they don't know which model made which, we'll call that a win.

Data

We went with the MS-COCO 2017 validation set. It has 5,000 everyday photos, and each one has five captions written by people. We looked at Flickr30k too, but COCO has a much wider mix of objects and scenes, and almost every text-to-image paper reports results on it, which makes it easier to compare our numbers against theirs. The validation split is also only about 1 GB, while the full training set is 18 GB, which we didn't need.

The captions needed some cleanup. Some were repeated, some were very short, and some were too long for CLIP, which can only read 77 tokens. We fixed spacing and punctuation, removed the duplicates, and kept captions between 5 and 40 words. We also threw out images that were tiny or oddly stretched, and resized everything else to 512×512. In the end we had 4,926 image–caption pairs: 4,434 to train on and 492 set aside for testing.

How we're approaching it

Stable Diffusion v1.5 is our base model. It relies on CLIP to understand the prompt, and it runs fine on the 8 GB laptop GPUs we have.

For this milestone we just wanted a baseline before touching anything. Given a caption, CLIP picked the right photo out of 492 about 67% of the time, and the right one showed up in its top five 91% of the time. We also generated five test images. Each took around 5 seconds, and they scored 28.9 on average on CLIPScore, a measure of how well an image matches its text. Those images are also where we spotted the missing remote and the one-deck bus.

Next we'll fine-tune the model on our training captions with LoRA. It only updates a small slice of the model (around 3 million parameters), so training fits on our laptops. After that we'll try a few different settings, like how strictly the model follows the prompt, how many steps it takes, and which caption it learns from, to see what actually matters. To judge the results we'll use CLIPScore, FID on the 492 test images, retrieval accuracy, and a quick survey where classmates pick whichever image fits the caption better.

Who's doing what

Member	What they're handling
Gokul Chandar Kumaran	The model and fine-tuning, GPU setup, the GitHub repo, and pulling the milestone reports together
Loalith Madala	The dataset: downloading and cleaning it, looking through the stats, and caption augmentation
Yagna Praneeth Reddy Panguru	Evaluation: the scoring scripts, the human survey, the result charts, and the demo

We'll write the final report and put the presentation together as a team.

Where things stand

Everything for Milestone 1 is up on GitHub: the data cleaning scripts, the environment setup, the baseline notebook, and our first five generated images. Our biggest worry is GPU memory. We're keeping that in check by running the model at half precision and using LoRA instead of retraining the whole thing. COCO captions can be hit or miss, so we filtered them and kept the best one for each photo. And since the COCO images come with their own licenses, we're only using them for this class project.